Automation, RPA & Advanced Excel — Hands-On

Hands-On Complete-Knowledge Deep-Dive (India, 2026)

Do-the-task drills: worked examples, entries, interview drills & free practice

Contents

Advanced Excel Power User: Dynamic Arrays, LAMBDA, LET, Data Model

What you'll be able to do

Turn Excel from a grid you type into by hand into a small application. By the end you can: write a single formula that spills a filtered, sorted, de-duplicated table; replace nested VLOOKUPs with XLOOKUP and dynamic-array logic; name your own reusable functions with LAMBDA so a 200-cell workbook shrinks to 20; build a proper relational Data Model (Power Pivot) across fact and dimension tables and write DAX measures; run reverse-solve tools (Goal Seek, Solver, Data Tables) for finance scenarios; and drive the whole thing keyboard-only. You'll build a working "Receivables Ageing + Collections" mini-tool that a manager could actually use.

The essentials

Dynamic arrays (Excel 365 / 2021+). One formula returns many results that "spill" into neighbouring cells. The top-left cell holds the formula; the spilled range is referenced with `#` (e.g. `E2#`). A `#SPILL!` error means something blocks the spill area — clear it.

Function	Does	Signature
`FILTER`	keep rows meeting a condition	`FILTER(array, include, [if_empty])`
`SORT` / `SORTBY`	order results	`SORT(array,[col],[order])`
`UNIQUE`	distinct values	`UNIQUE(array)`
`XLOOKUP`	lookup left/right, exact by default, returns arrays	`XLOOKUP(lookup, lookup_array, return_array, [if_not_found], [match_mode])`
`SEQUENCE`	generate 1..n	`SEQUENCE(rows,[cols],[start],[step])`

LET names intermediate calculations inside one formula so you compute once and read clearly: `LET(name1, value1, name2, value2, …, result)`. It's faster (no repeated sub-expressions) and readable.

LAMBDA defines a reusable function. Wrap your logic, name it in Name Manager (Formulas → Define Name), then call it like a built-in. Helpers: `MAP`, `REDUCE`, `SCAN`, `BYROW`, `BYCOL` apply a LAMBDA across arrays.

SUMPRODUCT multiplies arrays element-wise then sums — the classic pre-365 way to do conditional maths and weighted averages: `SUMPRODUCT((region="West")*(qty)*(price))`. Coercing booleans with `--` or `*` turns TRUE/FALSE into 1/0.

Data Model / Power Pivot. Load tables into the in-memory model (Data → "Add to Data Model" or from Power Query). Create *relationships* (one-to-many, dimension→fact) so you never VLOOKUP again. Write **DAX** measures: `Total Sales := SUMX(Sales, Sales[Qty]*Sales[Price])`. A PivotTable built on the model can slice a 5-million-row fact table instantly.

What-If tools. *Goal Seek* (Data → What-If → Goal Seek): set one cell to a target by changing one input. *Data Table*: recompute a formula across one or two varying inputs (sensitivity grid). *Solver* (add-in): optimise a target subject to multiple constraints — used for portfolio weights, capex allocation, blending.

Hands-on — step by step

Worked example: a raw invoice list on sheet `Data`, columns A–E: `Customer, Invoice, InvDate, Amount, PaidFlag`. Today is 13-Jul-2026. Sample rows: Alpha 001 10-Mar 120000 N; Beta 002 01-Jun 80000 N; Alpha 003 20-Jun 50000 Y; Gamma 004 05-May 200000 N.

1. **Make it a Table.** Click inside data → Ctrl+T → tick "My table has headers" → name it `tblInv` (Table Design → Table Name). Structured references now work: `tblInv[Amount]`.
2. **Open items only, sorted, one formula.** On sheet `Report` cell A1:

```
=SORT(FILTER(tblInv[[Customer]:[Amount]], tblInv[PaidFlag]="N", "No dues"), 4, -1)
```

This spills every unpaid invoice, largest first. No manual filter, no copy-paste.

3. **Ageing bucket with LET.** In a helper column of the model (or a spilled column) compute days overdue and bucket:

```
=LET(days, TODAY()-tblInv[InvDate],
     bucket, IFS(days≤30,"0-30", days≤60,"31-60", days≤90,"61-90", TRUE,"90+"),
     bucket)
```

4. **Reusable ageing function with LAMBDA.** Formulas → Name Manager → New. Name `AGEBUCKET`, Refers to:

```
=LAMBDA(d, IFS(d≤30,"0-30", d≤60,"31-60", d≤90,"61-90", TRUE,"90+"))
```

Now anywhere: `=AGEBUCKET(TODAY()-C2)` returns `61-90` for the 10-Mar invoice.

5. **Unique customers + outstanding per customer.**

```
=UNIQUE(FILTER(tblInv[Customer], tblInv[PaidFlag]="N"))
```

spills Alpha, Beta, Gamma in G2#. Beside it in H2:

```
=SUMIFS(tblInv[Amount], tblInv[Customer], G2#, tblInv[PaidFlag], "N")
```

Alpha 120000, Beta 80000, Gamma 200000. (SUMIFS accepts the spilled array and returns a spilled result.)

6. **SUMPRODUCT weighted DSO check.** Weighted average age of receivables:

```
=SUMPRODUCT((TODAY()-tblInv[InvDate])*(tblInv[PaidFlag]="N")*tblInv[Amount]) / SUMPRODUCT((tbl
```

7. **Data Model version.** Select `tblInv` → Data → "Add to Data Model". Add a `dimCustomer` table (Customer, Segment), add to model, then Power Pivot → Diagram View → drag `dimCustomer[Customer]` to `tblInv[Customer]` to make the relationship. In Power Pivot write measures:

```
Outstanding := CALCULATE(SUM(tblInv[Amount]), tblInv[PaidFlag]="N")
```

Build a PivotTable: Segment on rows, `[Outstanding]` in values, slicer by ageing bucket.

8. **Goal Seek scenario.** Cell B20 = target collection this month formula. Data → What-If → Goal Seek: Set B20 to 300000 by changing collection-rate cell B18. Excel solves the rate.
9. **Two-variable Data Table.** Lay a grid: collection rate down the left, discount % across the top, top-left cell = the net-cash formula. Select the block → Data → What-If → Data Table → row input = discount cell, column input = rate cell.

Keyboard-only speed: Ctrl+T (table), Ctrl+Shift+L (filter), Alt+= (AutoSum), Ctrl+Shift+↓ (select to end), Ctrl+` (show formulas), F4 (toggle $ absolute / repeat last action), Alt,A,W,G (Goal Seek), Ctrl+Shift+Enter is *no longer needed* for arrays in 365.

The output

A one-screen tool on `Report`:

```
RECEIVABLES AGEING — as at 13-Jul-2026
Customer   Outstanding   Bucket
Gamma        200,000     61-90
Alpha        120,000     90+
Beta          80,000     0-30
──────────────────────────────
Total open   400,000
Weighted age   ~78 days (DSO proxy)
```

Everything spills from live formulas; change a `PaidFlag` from N to Y and every number, the unique list, and the pivot update instantly. The Data-Model PivotTable gives the same totals sliceable by segment.

Checks, gotchas & red flags

- `#SPILL!` = the spill range is blocked or a merged cell sits in the way. Never merge cells in a data area.
- **Total must tie:** `SUM(H2#)` of per-customer outstanding must equal `SUMIFS` total open (400,000). If not, a PaidFlag has stray spaces — wrap in `TRIM`.
- **XLOOKUP vs VLOOKUP:** XLOOKUP defaults to *exact* match; VLOOKUP defaults to *approximate* (TRUE) — the classic silent-wrong-number bug. Always set VLOOKUP's 4th arg to FALSE.
- **Data Model relationships must be one-to-many** on a *unique* dimension key. Duplicate customer names in `dimCustomer` break the relationship (ambiguous).
- **Data Tables recalc the whole column** — slow on big models; switch calculation to "Automatic except data tables" (Formulas → Calc Options).

- **Solver** needs the add-in enabled (File → Options → Add-ins → Manage Excel Add-ins → Solver). It can return a *local* optimum; try different starting weights.
- Volatile functions (`TODAY` , `OFFSET` , `INDIRECT`) recalc on every change — avoid in huge sheets.

Interview drill

Q: When would you use LET over just writing the formula out? A: When a sub-expression repeats or the formula is unreadable. LET computes each named value once, so a formula that references `TODAY()-InvDate` three times evaluates it once — faster and self-documenting. It's the readability/performance bridge before you graduate to a named LAMBDA.

Q: You have a 4-million-row transaction file and need sales by region by month. Excel or Power Pivot, and why? A: Power Pivot / Data Model. The columnar in-memory engine compresses and aggregates millions of rows a normal grid can't hold (1,048,576-row limit), relationships replace lookups, and a DAX measure like `SUMX` computes on the fly. A regular PivotTable off raw rows would be slow and hit the row cap.

Q: Difference between SUMIFS and SUMPRODUCT for conditional sums? A: SUMIFS is faster and clearer for straightforward AND conditions on ranges. SUMPRODUCT is the tool when you need OR logic, arithmetic *inside* the condition (weighted averages, `days*amount`), or must support pre-365 files without dynamic arrays.

Learn/practise (free)

All of this works in the free web/desktop trial of Microsoft 365; if you only have Excel 2016 you'll miss dynamic arrays and LAMBDA — practise those in **Excel for the web** (free with a Microsoft account) which has them. ExcelJet's function reference and Chandoo.org are excellent free tutorials. For DAX, Microsoft Learn's "Power Pivot" path and SQLBIT (sqlbi.com) free articles. Rehearse by downloading any public CSV (e.g. NSE bhavcopy, RBI datasets) and rebuilding the ageing tool above; then reproduce the same report three ways — dynamic arrays, SUMPRODUCT, and a Data-Model PivotTable — to prove they tie out.

VBA Macros for Finance

What you'll be able to do

Automate the repetitive month-end grind: record a macro, then read and rewrite its code so it's robust; declare variables, loop over rows and files, make decisions with `If` / `Select Case`, pop message boxes, write your own worksheet functions (UDFs), and handle errors so a macro fails gracefully instead of crashing. You'll build two real tools shown in full: (1) a **format-and-email report** macro that cleans a sheet and sends it via Outlook, and (2) a **two-ledger reconciliation** macro that matches transactions and flags breaks. This is the skill that turns "I spend Friday afternoon reformatting" into "I press a button."

The essentials

Where code lives. Press **Alt+F11** to open the VBA editor (VBE). Code sits in *Modules* (Insert → Module) for general macros, or behind sheets/ThisWorkbook for event code. Save as **.xlsm** (macro-enabled) — .xlsx silently drops macros. Enable Developer tab: File → Options → Customize Ribbon → tick Developer. Set macro security: Trust Center → Macro Settings → "Disable with notification."

Core syntax. | Concept | Example | |---|---| | Procedure | `Sub DoThing()` … `End Sub` | | Variable | `Dim total As Double`, `Dim ws As Worksheet` | | Object refs | `Set ws = ThisWorkbook.Sheets("Data")` | | Cell/range | `ws.Range("B2").Value`, `ws.Cells(r, 2)` | | Loop | `For r = 2 To lastRow … Next r` | | Condition | `If amt > 0 Then … ElseIf … Else … End If` | | Message | `MsgBox "Done: " & n & " rows"` | | Comment | `' this is a comment` |

Find the last row (never hard-code): `lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row`.

Speed switches. Wrap heavy macros in `Application.ScreenUpdating = False` and `Application.Calculation = xlCalculationManual`, restore both at the end — often 10x faster.

UDF = a `Function` you call from a cell: `=GSTAMT(A2)`. It must return a value and live in a module.

Error handling. `On Error GoTo Handler` jumps to a label on failure; `On Error Resume Next` skips (use sparingly). Always restore app settings in the handler.

Hands-on — step by step

Step 1 — Record, then read. Developer → Record Macro, name `CleanFmt`. Bold the header row, autofit columns, add thousands separators to column D, stop recording. Alt+F11 to see the generated code — it's verbose and full of `.Select`. You'll learn by cleaning it: remove `.Select` / `Selection`, reference ranges directly.

Step 2 — Reconciliation macro (full code). Two sheets: `Ledger` (our books) and `Bank` (statement), each with `Ref` in col A and `Amount` in col B. Match on Ref, compare amounts, write a status.

```
Sub ReconcileLedgers()
    Dim wsL As Worksheet, wsB As Worksheet, wsR As Worksheet
    Dim lastL As Long, lastB As Long, r As Long
    Dim ref As String, ledAmt As Double, bnkAmt As Variant
    Dim matchRow As Variant, n As Long

    On Error GoTo Handler
    Application.ScreenUpdating = False

    Set wsL = ThisWorkbook.Sheets("Ledger")
    Set wsB = ThisWorkbook.Sheets("Bank")
    ' create/refresh a Results sheet
    On Error Resume Next
    Application.DisplayAlerts = False
    ThisWorkbook.Sheets("Recon").Delete
    Application.DisplayAlerts = True
    On Error GoTo Handler
    Set wsR = ThisWorkbook.Sheets.Add
    wsR.Name = "Recon"
    wsR.Range("A1:D1").Value = Array("Ref", "Ledger", "Bank", "Status")

    lastL = wsL.Cells(wsL.Rows.Count, "A").End(xlUp).Row
    n = 1
    For r = 2 To lastL
        ref = Trim(CStr(wsL.Cells(r, "A").Value))
        ledAmt = wsL.Cells(r, "B").Value
        ' look up ref in Bank col A
        matchRow = Application.Match(ref, wsB.Columns("A"), 0)
        n = n + 1
        wsR.Cells(n, 1).Value = ref
        wsR.Cells(n, 2).Value = ledAmt
        If IsError(matchRow) Then
            wsR.Cells(n, 3).Value = "—"
            wsR.Cells(n, 4).Value = "MISSING IN BANK"
        Else
            bnkAmt = wsB.Cells(matchRow, "B").Value
            wsR.Cells(n, 3).Value = bnkAmt
            If Abs(ledAmt - bnkAmt) < 0.01 Then
                wsR.Cells(n, 4).Value = "MATCHED"
            Else
                wsR.Cells(n, 4).Value = "DIFF " & Format(ledAmt - bnkAmt, "#,##0.00")
            End If
        End If
    Next r
```

```
    ' colour the breaks red
    Dim lastR As Long
    lastR = wsR.Cells(wsR.Rows.Count, "A").End(xlUp).Row
    For r = 2 To lastR
        If wsR.Cells(r, 4).Value <> "MATCHED" Then
            wsR.Range("A" & r & ":D" & r).Interior.Color = RGB(255, 199, 206)
        End If
    Next r
    wsR.Columns("A:D").AutoFit

    Application.ScreenUpdating = True
    MsgBox "Reconciliation done: " & (lastR - 1) & " items checked.", vbInformation
    Exit Sub

Handler:
    Application.ScreenUpdating = True
    Application.DisplayAlerts = True
    MsgBox "Error " & Err.Number & ": " & Err.Description, vbCritical
End Sub
```

Step 3 — Format-and-email report (full code). Uses Outlook (installed on most GCC desktops). Late binding, no reference needed.

```
Sub EmailReport()
    Dim ws As Worksheet, olApp As Object, mail As Object
    Dim lastRow As Long, total As Double, r As Long
    On Error GoTo Handler
    Set ws = ThisWorkbook.Sheets("Report")
    ws.Range("A1:D1").Font.Bold = True
    ws.Columns("A:D").AutoFit
    lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row
    For r = 2 To lastRow
        total = total + ws.Cells(r, 4).Value
    Next r
    Set olApp = CreateObject("Outlook.Application")
    Set mail = olApp.CreateItem(0)   ' 0 = mail item
    mail.To = "controller@company.com"
    mail.Subject = "Daily Report " & Format(Date, "dd-mmm-yyyy")
    mail.Body = "Hi," & vbCrLf & vbCrLf & _
        "Total for today: INR " & Format(total, "#,##0.00") & _
        " across " & (lastRow - 1) & " lines." & vbCrLf & _
        "Regards," & vbCrLf & "Finance Automation"
    mail.Display    ' use .Send to send automatically
    MsgBox "Draft created, total INR " & Format(total, "#,##0"), vbInformation
    Exit Sub
Handler:
    MsgBox "Error " & Err.Number & ": " & Err.Description, vbCritical
End Sub
```

Step 4 — A UDF. In a module:

```
Function GSTAMT(base As Double, Optional rate As Double = 0.18) As Double
    GSTAMT = base * rate
End Function
```

Use `=GSTAMT(A2)` → 18% GST, or `=GSTAMT(A2, 0.12)` for 12%.

Step 5 — Run & bind. Press F5 in VBE to run, or add a button: Developer → Insert → Button (Form Control) → assign `ReconcileLedgers`.

The output

Running `ReconcileLedgers` produces a fresh **Recon** sheet:

```
Ref     Ledger      Bank        Status
INV001  120,000.00  120,000.00  MATCHED
INV002   80,000.00   79,500.00  DIFF 500.00      ← red
INV003   50,000.00   –          MISSING IN BANK  ← red
```

and a popup "Reconciliation done: 3 items checked." `EmailReport` opens a pre-filled Outlook draft with the day's total. Both are one-click, repeatable, and leave an auditable coloured trail.

Checks, gotchas & red flags

- **Save as .xlsm.** Saving as .xlsx destroys every macro without warning.
- **Never leave** `.Select` / `Activate` from recorded code — it's slow and breaks when the wrong sheet is active. Reference objects directly.
- **Floating-point:** compare amounts with `Abs(a-b) < 0.01`, never `a = b` — 79.5 stored as 79.4999999 will falsely flag.
- `Application.Match` **returns an error**, not zero, on no-match — test with `IsError`, don't assume.
- **Restore settings in the handler:** if a macro dies with `ScreenUpdating=False`, the screen freezes until you re-run. Always reset in the error path.
- **Auto-send is dangerous:** keep `.Display` (draft) until tested; only switch to `.Send` when the recipient and content are verified. A bad loop can email 500 people.
- **Trim and CStr** keys before matching — trailing spaces and number-vs-text mismatches are the #1 recon false-break.

Interview drill

Q: How do you make a VBA macro run faster on 100k rows? A: Turn off `ScreenUpdating`, set `Calculation` to manual, and disable `EnableEvents`; read the range into a Variant array (`arr = ws.Range( ... ).Value`), process in memory, write back in one shot. Avoid `.Select`, and restore all settings at the end and in the error handler.

Q: Difference between a Sub and a Function in VBA? A: A `Sub` performs actions and returns nothing; you run it from a button or F5. A `Function` returns a value and can be called from a worksheet cell as a UDF or from other code. Recon/email are Subs; `GSTAMT` is a Function.

Q: How do you handle errors so a finance macro doesn't crash mid-run? A: `On Error GoTo Handler` at the top, a labelled `Handler:` block at the end that reports `Err.Number` / `Err.Description` and restores `ScreenUpdating`, `Calculation`, and `DisplayAlerts`. Use `On Error Resume Next` only around a single known-safe line (like deleting a sheet that may not exist), then immediately reinstate `On Error GoTo Handler`.

Learn/practise (free)

Everything here runs in any licensed desktop Excel — no extra tools. Free learning: Excel Macro Mastery (excelmacromastery.com) is the best VBA reference; Chandoo and WiseOwl (YouTube) have full free courses. Microsoft's VBA language reference on Microsoft Learn documents every object/method. Practise by recording a macro for any manual task you do, then opening it and deleting all `.Select` lines until it still works — that single exercise teaches the object model faster than any tutorial. Rebuild the reconciliation above with your own two CSVs to internalise `Match`, `IsError`, and the last-row pattern.

Advanced Power Query & Power Automate

What you'll be able to do

Build a *refreshable* data pipeline instead of copy-pasting. In Power Query you'll go past "remove columns": merge and append queries, add custom columns in M, parameterise a file path or period, unpivot a crosstab into a tidy table, and group/aggregate — all as steps that re-run at the click of Refresh. Then in Power Automate (cloud flows) you'll schedule a report refresh, route an approval, and trigger an email/Teams message on an event using connectors. The worked example: a monthly **branch sales consolidation** that pulls many files, cleans them, and fires an approval + Teams alert when a branch is over budget.

The essentials

Power Query (Get & Transform) lives in Excel (Data → Get Data) and Power BI. Every click writes a step in the **Applied Steps** pane; the underlying language is **M** (case-sensitive, functional). Queries *load* to a table, the Data Model, or "connection only."

Key operations: | Task | Where | Note | |---|---|---| | **Append** | Home → Append Queries | stack tables with same columns (Jan+Feb+Mar) | | **Merge** | Home → Merge Queries | join on key(s); Left Outer / Inner / Anti | | **Custom column** | Add Column → Custom | write an M expression | | **Unpivot** | Transform → Unpivot Columns | turn month-columns into rows | | **Group By** | Transform → Group By | sum/count per key | | **Parameters** | Manage Parameters | reusable inputs (folder, period) |

Merge join kinds: Left Outer (all left + matches), Inner (only matches), **Left Anti** (rows in left with *no* match — perfect for "unmatched/missing" recon).

M basics. A query is `let … in` : `let Source = …, Step2 = Table.SelectRows(Source, each [Amount] > 0) in Step2` . `each` is shorthand for a function; `[Column]` references a field. Custom column example: `if [Region]="West" then [Sales]*1.1 else [Sales]` .

From Folder (Data → Get Data → From File → From Folder) reads *every* file in a folder and combines them — drop next month's file in and Refresh; no rebuild.

Power Automate (flow.microsoft.com) runs *cloud flows*. A flow = **trigger** + **actions**. - Triggers: *Recurrence* (scheduled), *When a new email arrives*, *When a file is created* (SharePoint/OneDrive), *When an item is created* (Lists), manual button. - Actions via **connectors**: Outlook/Exchange, Teams, SharePoint, Excel Online (Business), Approvals, HTTP, and the "Dataflow/Power BI refresh" connectors. - **Approvals**: "Start and wait for an approval" pauses the flow until someone clicks Approve/Reject; you branch on the outcome. - Dynamic content passes data between steps; **Condition** and **Apply to each** give logic and loops.

Licensing note: Power Query is free (built into Excel/Power BI Desktop). Power Automate *cloud flows* need a Microsoft 365 work/school account; the standard connectors (Outlook, Teams, SharePoint, Excel Online, Approvals) are included in most M365 business plans. Premium connectors (SQL Server on-prem via gateway, Dataverse, HTTP) may need a per-user/per-flow plan.

Hands-on — step by step

Part A — Consolidate branch files in Power Query. 1. Put `Delhi.xlsx` , `Mumbai.xlsx` , `Chennai.xlsx` in one folder, each with a sheet `Sales` shaped as a crosstab: `Product | Jan | Feb | Mar` . 2. Excel → Data →

Get Data → From File → **From Folder** → pick the folder → **Combine & Transform**. Power Query shows a sample; it auto-generates a function that opens each file's `Sales` sheet and appends them, adding a `Source.Name` column. 3. **Clean the branch name:** Add Column → Custom Column, name `Branch`:

```
Text.BeforeDelimiter([Source.Name], ".")
```

4. **Unpivot months:** select `Jan, Feb, Mar` → Transform → **Unpivot Columns**. Rename `Attribute` → `Month`, `Value` → `Sales`. You now have tidy rows: Branch, Product, Month, Sales.
5. **Parameter for period.** Home → Manage Parameters → New, name `pPeriod`, type Text, current value `Mar`. Then Home → Keep Rows / filter `Month` = `pPeriod`, but change the filter step's formula to reference the parameter:

```
= Table.SelectRows(#"Renamed", each [Month] = pPeriod)
```

Change `pPeriod` once and every downstream number moves to a different month. 6. **Merge a budget.** Load a `Budget` query (Branch, Month, Budget). Home → **Merge Queries** → match on Branch *and* Month (Ctrl-click both) → Left Outer → expand `Budget`. 7. **Variance custom column** `Variance`: `= [Sales] - [Budget]`, and `OverBudget`: `if [Sales] > [Budget] then "YES" else "NO"`. 8. **Group By** (a second query, reference the cleaned one): Transform → Group By → group on Branch → New column `TotalSales` = Sum of `Sales`. 9. Home → **Close & Load To** → Table (or "Only Create Connection" + Add to Data Model). Next month: drop the new file in the folder, hit **Refresh All** — done.

Part B — Power Automate flow: refresh + approval + alert. 1. Save the workbook to **SharePoint/OneDrive for Business** (cloud flows can only reach cloud files) as a Table named `tblConsol`. 2. Go to flow.microsoft.com → **Create** → **Scheduled cloud flow**. Name "Monthly Sales Consolidation", Recurrence: 1st of month, 09:00. 3. Action: **Excel Online (Business)** → **List rows present in a table** → point to the workbook and `tblConsol`. 4. Action: **Apply to each** over the rows. Inside, add a **Condition**: `OverBudget is equal to YES`. 5. In the *If yes* branch: **Approvals** → **Start and wait for an approval** → type "Approve/Reject", Title = `Branch @{items('Apply_to_each')?['Branch']} over budget`, assigned to the finance controller, Details = variance amount via dynamic content. 6. After the approval, add a **Condition** on `Outcome = Approve`. If approved → **Post message in a chat or channel (Teams)** to the #finance channel: "Overspend at @{...Branch} of INR @{...Variance} approved." If rejected → **Send an email (V2)** back to the branch manager asking for justification. 7. Add a final **Send an email (V2)** to the controller with the run summary. **Save** → **Test** → **Manually** to dry-run.

The output

Power Query result table (loaded to the sheet, fully refreshable):

```
Branch    Product   Month  Sales    Budget   Variance  OverBudget
Delhi     Widget    Mar    520,000  500,000   20,000   YES
Mumbai    Widget    Mar    460,000  500,000  -40,000   NO
Chennai   Widget    Mar    610,000  550,000   60,000   YES
```

Power Automate run: two approval requests (Delhi, Chennai) land in the controller's Approvals hub and Outlook; on approval a Teams message posts to #finance and a summary email is sent. The whole month-end consolidation now runs itself on the 1st at 9am with a human gate only where money is over budget.

Checks, gotchas & red flags

- **Refresh must reproduce, not accumulate.** From-Folder appends *all* files present — if last month's file is still there you'll double-count. Keep the folder to the current set or add a date filter.
- **Data types before load.** Power Query is case-sensitive and type-strict; set each column's type (the little icon) so `Sales` sums instead of erroring. A wrong type on a merge key → zero matches.
- **Merge key must be clean on both sides** — trailing spaces or "Delhi " vs "Delhi" silently produce nulls (use Left Anti join to *see* the unmatched rows).
- **Cloud flows can't reach local files.** The workbook must live in SharePoint/OneDrive Business; a desktop path fails.
- `Apply to each` **on thousands of rows is slow and can hit action limits** — filter rows *before* the loop (use "List rows" with a filter query) rather than looping everything.
- **Approvals need valid work emails**; external/guest approvers may need extra config.
- **Timezone:** Recurrence uses UTC by default — set the timezone or your 09:00 fires at 14:30 IST.
- Don't hard-code the period inside a step *and* in the parameter — change it in one place only.

Interview drill

Q: Merge vs Append in Power Query — when each? A: Append stacks tables with the *same columns* vertically (combining Jan, Feb, Mar branch files into one longer table). Merge joins tables *side by side* on a key to bring columns together (attaching Budget to Sales on Branch+Month). Recon uses a Left Anti merge to isolate rows with no match.

Q: How would you build a report that a colleague can update with zero Excel skill? A: Power Query From-Folder into a parameterised query loaded to a Table/Data Model. They just drop the new file in the folder and press Refresh All — every clean, merge, unpivot and variance step re-runs deterministically. Optionally schedule a Power Automate flow to refresh and email it so they don't even open Excel.

Q: What's the trigger-plus-action model in Power Automate, and give a finance example? A: A cloud flow starts on a *trigger* (a schedule, a new email, a file created) and then runs *actions* through connectors. Example: Recurrence trigger on the 1st → List rows from an Excel table → for over-budget rows, Start-and-wait-for-approval → on approve, post to Teams and email the controller. It's event-driven RPA without code.

Learn/practise (free)

Power Query is free in Excel and in **Power BI Desktop** (free download) — practise M there with the "Advanced Editor." Microsoft Learn has full free paths: "Get and transform data in Power BI" and "Automate a business process using Power Automate." For Power Automate, the personal Microsoft 365 developer/trial tenant gives a free work account with standard connectors; or use the **free Power Automate plan** for basic flows. Ken Puls / Excelguru and "Curbal" (YouTube) teach M superbly for free. Rehearse by taking three messy CSVs, combining From-Folder, unpivoting, merging a budget, and confirming the grouped total ties to the raw sum — then schedule a trial flow that emails you the result.

Alteryx for Finance

What you'll be able to do

Build a **repeatable, self-documenting data-prep workflow** in Alteryx Designer that a finance team runs every close without touching a formula. You'll know the canvas and the core tools — Input, Select, Filter, Formula, Join, Summarize, Output — and chain them into a visual pipeline that reconciles two ledgers and consolidates multi-entity data. You'll be able to explain *when Alteryx beats Excel* (and when it doesn't), read a workflow someone else built, and produce a clean output file plus a report of breaks. This is the tool GCC finance-transformation teams use to retire fragile macro-and-VLOOKUP spreadsheets.

The essentials

Alteryx Designer is a desktop drag-and-drop ETL tool. You place **tools** on a **canvas** and connect them with wires; data flows left→right. Each tool shows a config panel and, at runtime, row counts on every wire — so you can see exactly where records drop. A saved workflow is a `.yxmd` file; run it and it reproduces identically every time. No coding required, though it supports formulas and R/Python.

The core tool palette (the 80% you'll use): | Tool | Palette | Does | |---|---|---| | **Input Data** | In/Out | read a file/DB (xlsx, csv, SQL, etc.) | | **Output Data** | In/Out | write result to file/DB | | **Select** | Preparation | keep/drop/rename columns, set data types | | **Filter** | Preparation | split rows by a condition (True/False outputs) | | **Formula** | Preparation | add/modify columns with expressions | | **Sort** | Preparation | order rows | | **Join** | Join | match two inputs on key → **L** (left-only), **J** (matched), **R** (right-only) outputs | | **Union** | Join | stack inputs (append) | | **Summarize** | Transform | group & aggregate (sum, count, min…) | | **Unique** | Preparation | de-duplicate |

The Join tool is the heart of reconciliation. Its three anchors are gold: **J** = matched pairs, **L** = records only in the left source, **R** = records only in the right. In recon, L and R *are* your breaks — no formula needed.

Formula language resembles Excel: `IF [Sales] > [Budget] THEN "Over" ELSE "OK" ENDIF`, `ABS([Ledger]-[Bank])`, `Trim([Ref])`, `DateTimeToday()`.

When Alteryx beats Excel: repeatable monthly processes; combining many files/sources; data too big for a sheet; when you need an *auditable* pipeline (every step visible, row counts prove nothing was lost); when handing a process to a team so it doesn't depend on one person's macro. **When Excel wins:** one-off analysis, ad-hoc modelling, final presentation formatting, or when no one on the team has an Alteryx licence (it's expensive — see below).

Hands-on — step by step

Goal: reconcile **Ledger.xlsx** (our books: Ref, Amount) against **Bank.xlsx** (statement: Ref, Amount), flag amount differences and one-sided items, and output a clean workbook.

1. **Input Data ×2.** Drag two Input Data tools. Point the first at `Ledger.xlsx` (select the sheet), the second at `Bank.xlsx`. Run once (Ctrl+R) — wires show 3 rows and 3 rows.
2. **Select on each.** Add a Select tool after each Input. Rename `Amount` → `LedgerAmt` on the ledger branch and `Amount` → `BankAmt` on the bank branch (so columns don't clash after the join). Confirm `Ref` is a String type and `*Amt` is Double.

3. **Clean the key.** Add a Formula tool on each branch: overwrite `Ref` with `Trim([Ref])` to kill trailing spaces (the classic false-break cause).
4. **Join.** Drag a Join tool; wire ledger to the **L** input, bank to the **R** input. In config, join on `Ref = Ref`. Run. Now: - **J** anchor = refs in both. - **L** anchor = refs only in the ledger → "*Missing in Bank.*" - **R** anchor = refs only in the bank → "*Missing in Ledger.*"
5. **Compare amounts (off the J anchor).** Add a Formula tool: new column `Diff` = `[LedgerAmt] - [BankAmt]`, and `Status` = `IF ABS([LedgerAmt]-[BankAmt]) < 0.01 THEN "MATCHED" ELSE "DIFF" ENDIF`.
6. **Label the one-sided items.** Off **L**, a Formula adds `Status` = `"MISSING IN BANK"`. Off **R**, a Formula adds `Status` = `"MISSING IN LEDGER"`.
7. **Union everything.** Drag a Union tool; wire the J-branch, L-branch and R-branch into it. It auto-aligns columns by name (fill gaps with null). You now have one table of every ref with a Status.
8. **Filter breaks.** Add a Filter: `[Status] ≠ "MATCHED"`. The **True** output = the breaks worth reviewing.
9. **Summarize a control total.** In parallel, add a Summarize off the ledger branch: Group nothing, Sum `LedgerAmt`; do the same for bank. This gives you a tie-out control (total ledger vs total bank).
10. **Output Data.** Wire the Union to an Output Data tool → write `Recon_Output.xlsx`, sheet `AllItems`; wire the Filter-True to a second Output → sheet `Breaks`. Save the workflow as `LedgerRecon.yxmd`.
11. **Re-run next month:** replace the two input files (same names) and press Ctrl+R. Identical logic, new numbers, in seconds.

Consolidation variant: to consolidate five entity files, use **Input** (or a single Input with a wildcard `*.xlsx` and "Output File Name as Field"), a **Select** to standardise columns, a **Union** to stack them, a **Formula** to tag the entity from the filename, then **Summarize** grouping by Account to get the consolidated trial balance.

The output

`Recon_Output.xlsx` → sheet **AllItems**:

```
Ref     LedgerAmt   BankAmt    Diff     Status
INV001   120000     120000        0     MATCHED
INV002    80000      79500      500     DIFF
INV003    50000       (null)      -     MISSING IN BANK
INV009    (null)      30000       -     MISSING IN LEDGER
```

sheet **Breaks** (Filter-True) shows only INV002, INV003, INV009 — the exact list a reviewer works. The workflow canvas itself is the documentation: anyone can open `.yxmd` and see Input → Select → Formula → Join → Union → Filter → Output with live row counts proving 3+3 records fully accounted for.

Checks, gotchas & red flags

- **Row-count reconciliation:** L-count + J-count must equal the left input count; R + J must equal the right. If not, a null or type-mismatch in the key silently dropped rows.
- **Join keys must be same data type.** `Ref` as String on one side and Int64 on the other → zero matches. Set types in Select first.
- **Trim keys** on both sides — trailing spaces are the number-one false break; do it before the Join, not after.

- **Union aligns by name by default;** if a source misspells a column it lands in a new column full of nulls. Check the Union config's field map.
- **Float comparison:** use `ABS(a-b) < 0.01`, never `a = b`.
- **Don't confuse Join's L/R anchors** — L is the *top* input (unmatched-left), R the bottom. Mislabelling flips "missing in bank" vs "missing in ledger."
- **Licensing red flag:** Alteryx Designer is a paid annual licence (roughly USD 5k+ per user/year list). Don't propose it for a one-analyst one-off — that's where Excel/Power Query wins. It earns its cost on recurring, multi-source, audited team processes.

Interview drill

Q: How do you reconcile two ledgers in Alteryx without a single formula for matching? A: A Join tool on the reference key. The J anchor gives matched pairs (then a Formula compares amounts), the L anchor gives items only in the first source, the R anchor items only in the second. L and R are the one-sided breaks by construction — the tool separates them for you; I only add a status label and Union them back for reporting.

Q: When would you choose Alteryx over Excel/Power Query? A: When the process repeats every close, pulls from several files or databases, must be auditable (visible steps and row counts prove completeness), and is handed to a team rather than owned by one person's macro. For a one-off analysis or final formatting, Excel is faster and cheaper — and Power Query covers many mid-size repeatable jobs for free, so I'd only reach for Alteryx when scale, source variety, or governance justify the licence.

Q: A join returns far fewer matched rows than expected. How do you debug? A: Check the L and R anchors' row counts to see what didn't match, then inspect the key: data-type mismatch (String vs Int), casing, and trailing spaces are the usual causes. Add Trim/UpperCase Formulas before the Join and confirm both keys are the same type in the Select tool; re-run and verify L+J and R+J reconcile to the input counts.

Learn/practise (free)

Alteryx offers a **free 30-day Designer trial** and, importantly, **Alteryx Community** (community.alteryx.com) with the **Weekly Challenges** — dozens of free, graded data-prep puzzles with sample data and solution workflows; work through the beginner set to drill Join/Union/Summarize. The **Alteryx SparkED** programme gives students free learning licences and courses. If you can't get a licence, replicate every workflow above in **Power Query** (free) — the concepts map one-to-one (Merge = Join, Append = Union, Group By = Summarize), so you learn the pattern and can speak to Alteryx credibly in interviews. Rehearse by building the ledger recon end-to-end, deliberately introducing a trailing space and a type mismatch, and watching the L/R anchors reveal the breaks.

RPA in Finance: UiPath, Automation Anywhere, Blue Prism

What you'll be able to do

By the end of this chapter you can explain what Robotic Process Automation (RPA) actually is (and isn't), tell an interviewer the difference between an attended and an unattended bot, and — most importantly — build a working UiPath bot that automates a real finance process end-to-end: log into a bank portal (or open a downloaded statement), pull the day's transaction file, reconcile it against a ledger extract in Excel, flag the breaks, and post/write-back the matched items. You will know when RPA is the right tool versus an API or a macro, and you'll be able to speak to governance (bot IDs, credential vaults, maker-checker, audit logs) the way a GCC or shared-services hiring manager expects. You can rehearse all of this free on the UiPath Community Edition.

The essentials

RPA is software that mimics a human using a computer — it clicks, types, reads screens, opens files, copies cells — driven by a defined workflow. It sits *on top* of existing systems (SAP, a bank portal, Excel, a web ERP) and needs no change to those systems. That is its whole selling point in finance: the underlying apps often have no API, or IT will not sanction a database integration, but a bot can do exactly what the analyst did, only faster and without fatigue.

The three market leaders (mid-2026):

Tool	Position in India/GCC	Dev environment	Notable
UiPath	Market leader, most job postings, biggest India footprint	Studio (visual, VB/C# expressions) + Community free	Best learning path (UiPath Academy, free certs); AI/Document Understanding strong
Automation Anywhere	Strong in BPO/BFSI back-offices	Cloud-native "Automation 360" (browser-based Control Room)	Bot Store; good for high-volume unattended fleets
Blue Prism (SS&C)	Enterprise/bank IT-governed shops	"Process Studio" — no recorder, code-like, object/process split	Very governance-heavy; favoured where audit rules are strict

Attended vs unattended — know this cold:

- **Attended bot** runs on the user's own machine, triggered by the human (a button, a hotkey), works *alongside* them — e.g. a bot the AP clerk fires to auto-fill an invoice screen. Think co-pilot.
- **Unattended bot** runs on a server/VM with no human present, triggered by a schedule or a queue, at 2 a.m. — e.g. the nightly bank-reconciliation run. Needs an Orchestrator (UiPath) / Control Room (AA) to license, schedule, log and hand out credentials.

Core building blocks (UiPath vocabulary): *Workflow* (a .xaml file of steps), *Activity* (one step — Click, Type Into, Read Range), *Recorder* (watches you do a task and generates activities), *Selector* (the XML address of a

UI element the bot targets), *Orchestrator* (the web control plane: assets, queues, robots, schedules, logs), *Asset* (a stored config value or credential), *Queue* (a list of work items processed transactionally).

When RPA is the right tool — the decision rule:

Situation	Use
Target app has a clean, supported **API**	API integration (Python/Power Automate) — more robust, no screen fragility
It's all inside **Excel/Outlook**, one machine, simple	**VBA / Office Scripts / Power Query** — lighter, free, no bot licence
No API, multiple apps, GUI-only, repetitive, rule-based, high volume	**RPA** — this is its sweet spot
Process needs **judgement / changes every time**	Keep it human (or add AI-in-the-loop, carefully)

RPA is a bridge for legacy/GUI systems. If a real API exists, prefer it — bots break when a screen layout, a button label, or a login flow changes.

Hands-on — step by step

Worked example — daily bank reconciliation. Every morning you download an HDFC current-account statement (CSV), compare it against the ledger extract from Tally/SAP (Excel), and mark which bank lines match a book entry. Amounts in rupees.

`bank_statement.csv` :

Date	Description	Ref	Amount
2026-07-13	NEFT ACME LTD	N123	250000
2026-07-13	UPI VENDOR X	U987	-18500
2026-07-13	BANK CHG	—	-472

`ledger.xlsx` :

Ref	Party	BookAmount	Status
N123	Acme Ltd	250000	
U987	Vendor X	-18500	

Build it in UiPath Studio:

1. **New Process** → name it `DailyBankRecon` . Studio opens `Main.xaml` with an empty *Sequence*.
2. **Get the file.** For practice, drop the CSV into a `C:\Recon\in\` folder (in production a *Use Application/Browser* + *Type Into* + *Click* sequence, or a recorded login, downloads it from the portal — that's the "attended login" part). Add **Read CSV** activity → input `C:\Recon\in\bank_statement.csv` → output DataTable `dtBank` .
3. **Read the ledger.** Add **Excel Process Scope** → **Use Excel File** (`ledger.xlsx`) → **Read Range** → output `dtLedger` .

4. **Match.** Add a **For Each Row in DataTable** over `dtBank`. Inside, use an **Assign** to look up the ref in the ledger: `matchRow = dtLedger.AsEnumerable().FirstOrDefault(Function(r) r("Ref").ToString = CurrentRow("Ref").ToString)`
5. **Decide** with an **If**: - Condition: `matchRow IsNot Nothing AndAlso CDbl(matchRow("BookAmount")) = CDbl(CurrentRow("Amount"))` - **Then** (matched): Assign `matchRow("Status") = "MATCHED"`. - **Else** (break): add the row to a `dtBreaks` DataTable via **Add Data Row** (this is the exception list).
6. **Post / write back.** After the loop, **Write Range** `dtLedger` back to `ledger.xlsx` (Status column now populated) and **Write CSV** `dtBreaks` to `C:\Recon\out\breaks_2026-07-13.csv`.
7. **Notify.** Add **Send Outlook Mail Message** → to the accountant, subject "Bank recon 2026-07-13: 2 matched, 1 break", attach the breaks file.
8. **Log & credentials.** Wrap risky steps in **Try Catch**; use **Log Message** at each stage. Never hard-code the portal password — store it as an **Orchestrator Asset (Credential)** or Windows Credential and fetch with **Get Credential**.
9. **Run** (green ▶). Watch it step through. To schedule unattended, publish to **Orchestrator** and set a 7:30 a.m. trigger.

The **Recorder** (Ribbon → Recording → *App/Web*) shortcut: click "Record", perform the portal login and download manually once, hit Save — Studio generates the Click/Type Into activities and their selectors for you. You then clean up the selectors and wrap them in Try Catch.

The output

Console/Orchestrator log:

```
[INFO] DailyBankRecon started 07:30:01
[INFO] Read 3 bank rows, 2 ledger rows
[INFO] N123  250000  → MATCHED
[INFO] U987  -18500  → MATCHED
[WARN] BANK CHG -472  → BREAK (no ledger ref)
[INFO] Wrote breaks_2026-07-13.csv (1 row); ledger written back; mail sent
[INFO] Completed 07:30:14  duration 13s
```

`ledger.xlsx` (Status filled) and `breaks_2026-07-13.csv`:

Date	Description	Ref	Amount
2026-07-13	BANK CHG	—	-472

Deliverable = reconciled ledger + a one-line break (bank charges not yet booked — the accountant posts an expense entry) + an audit-logged, e-mailed run.

Checks, gotchas & red flags

- **Must tie out:** matched + breaks = total bank lines (2 + 1 = 3). If not, a row was dropped — check the loop and date/number formats.

- **Type traps:** CSV amounts come in as *text*. `CDbl()` them before comparing, and beware "18,500" (comma) or "(18500)" (brackets = negative) — strip/convert first.
- **Selector fragility (the #1 RPA failure):** dynamic IDs, changed labels, or a portal redesign break the bot silently. Use anchors, wildcards (`*`), and stable attributes; never rely on screen coordinates.
- **Idempotency:** if the bot re-runs, don't double-post. Check the Status flag before writing.
- **Credentials in plain text** in a variable or config = instant audit fail. Always use a vault/Asset.
- **No exception handling** = a single pop-up hangs the whole unattended run. Wrap in Try Catch and route failures to a queue.
- **Governance red flags:** no bot ID, no maker-checker on what it posts, no audit log, bot running under a personal login instead of a service account. Auditors will ask exactly these.

Interview drill

Q1. "Attended or unattended bot for month-end bank recon across 40 accounts — which and why?" Unattended. It's high-volume, rule-based, runs overnight with no human, and needs central scheduling, credential vaulting and audit logs — that's Orchestrator/unattended territory. Attended suits a human-triggered, single-desk task. I'd feed the 40 accounts through an Orchestrator *queue* so each is a transactional work item with retry and a clean per-account audit trail.

Q2. "The bank has a REST API. Would you still use RPA?" No — I'd integrate via the API (Python/requests or Power Automate). It's far more robust than screen-scraping, survives UI changes, and is easier to audit. RPA is my tool when there's *no* API and the system is GUI-only. Choosing RPA over an available API is a common anti-pattern that creates fragile automation.

Q3. "How do you make an RPA bank-recon auditable?" Service account (not a personal login), credentials in a vault/Asset, unique bot ID, Log Message at every stage into Orchestrator, a maker-checker so the bot proposes postings but a human approves the break entries, versioned workflow in Git, and an exception queue so nothing fails silently. I'd also reconcile control totals (matched + breaks = total) each run.

Learn/practise (free)

- **UiPath Community Edition** — free Studio download, full-featured for learning; **Community Orchestrator** (free cloud tenant) to practise assets, queues and schedules.
- **UiPath Academy** — free courses and the **UiPath Certified Professional – Automation Developer Associate (UiPath-ADAv1)** cert path; do the "RPA Developer Foundation".
- **Automation Anywhere University** and **Blue Prism Learning** — free community/learning editions to see the other two Control Rooms.
- **Rehearse cheaply:** you need no bank at all — generate a fake `bank_statement.csv`, build the recon above, then extend it: add a portal-login recording against any practice site (UiPath's own ACME test site), route breaks to a queue, and schedule an unattended run on your own machine. Put the .xaml on GitHub — that becomes a portfolio piece (next chapter).

Automation Portfolio & Interview Drills + Cheat-Sheet

What you'll be able to do

By the end of this chapter you can package everything from this guide into a single, showable automation portfolio project — a real finance process, automated end-to-end, documented so a hiring manager grasps the value in 30 seconds and a technical interviewer can inspect the code. You will be able to answer the standard "how would you automate this finance task?" question with a structured, credible walkthrough, defend your tool choices, and reel off a one-page cheat-sheet of the automation stack — which tool for which job — plus the Excel/VBA/Power Query shortcuts and functions that come up in live tests. This is the chapter that turns "I know some automation" into "here is the thing I built, here is the time it saves, here is the code."

The essentials

A portfolio project that lands interviews has five properties: it solves a **recognisable finance pain** (recon, MIS, AP, GST), it is **end-to-end** (input → process → output → notify), it shows a **before/after time saving** in numbers, it is **documented** (README + a 60-second demo GIF/video), and the **code is public** (GitHub) and clean. Interviewers don't reward cleverness; they reward *judgement* — did you pick the right tool, did you handle errors, can it be audited.

The reference project — "Daily MIS & Bank Recon Pack": it ingests a bank CSV and a ledger extract, reconciles them (Power Query or Python), refreshes a Power BI/Excel MIS dashboard, flags breaks, and e-mails a PDF pack to the finance head every morning. It legitimately touches every tool an analyst is hired for. You claim a real number: "manual = 45 min/day; automated = under 2 min; ~15 hours/month saved."

The "how would you automate X?" answer framework — always use these six beats:

1. **Clarify** the process, inputs, outputs, frequency, volume.
2. **Map** the as-is steps (a swimlane in words).
3. **Pick the tool** with the decision rule (below) and justify it.
4. **Handle exceptions** — what breaks, how you catch it, maker-checker.
5. **Govern** — credentials, audit log, service account, versioning.
6. **Measure** — time saved, error reduction, and what you'd automate next.

Hands-on — step by step

Build the portfolio repo:

1. **Create the GitHub repo** `finance-automation-recon`. Structure: `/data sample_bank.csv, sample_ledger.xlsx (fake data – never real client data) /pq recon.pq (Power Query M) /python recon.py (pandas alternative) /uipath DailyRecon.xaml /docs demo.gif, dashboard.png README.md`
2. **Write the Power Query version.** In Excel: Data → Get Data → From CSV (bank), From Workbook (ledger). Merge the two queries on `Ref` (Left Outer). Add a custom column `Status = if [BookAmount] =`

`[Amount] then "MATCHED" else "BREAK"`. Close & Load to a table. This is your no-code, auditable core.

3. **Add the Python version** (shows range): `python import pandas as pd bank = pd.read_csv("data/sample_bank.csv") led = pd.read_excel("data/sample_ledger.xlsx") m = bank.merge(led, on="Ref", how="left", indicator=True) m["Status"] = (m["Amount"] == m["BookAmount"]).map({True:"MATCHED", False:"BREAK"}) breaks = m[(m._merge=="left_only") | (m.Status=="BREAK")] breaks.to_csv("out/breaks.csv", index=False) print(f"{len(m)-len(breaks)} matched, {len(breaks)} breaks")`
4. **Add a dashboard.** A Power BI or Excel PivotChart: matched vs breaks by day, total value reconciled, top break reasons. Screenshot it to `/docs/dashboard.png`.
5. **Record a demo.** Screen-record the run (input file appears → click refresh → dashboard updates → email fires). Export a 30–60s GIF to `/docs/demo.gif` and embed it at the top of the README.
6. **Write the README** with: one-line pitch, the before/after time table, an architecture diagram (input→Power Query→dashboard→email), "tools used", "how to run", and "what I'd do next (queue-based unattended bot, API pull)".
7. **Commit clean history** — small, sensible commits; a `.gitignore` for temp files; no credentials, no real data.

The output

The README top section a recruiter sees:

```
# Daily MIS & Bank Recon Automation
Reconciles the bank statement against the ledger, refreshes the MIS
dashboard, and emails a break-report pack every morning — hands-free.

| Metric        | Manual   | Automated | Saving         |
|---------------|----------|-----------|----------------|
| Time / day    | 45 min   | < 2 min   | ~43 min        |
| Errors        | ~3/week  | 0         | eliminated     |
| Time / month  | 15 hrs   | 40 min    | ~14 hrs saved |

Tools: Power Query · Python (pandas) · Power BI · UiPath (unattended) · Outlook
![demo](docs/demo.gif)
```

Deliverable = a public repo with three working implementations, a dashboard image, a demo GIF, and a quantified value story — the single strongest thing you can put on a finance-analyst CV under "Projects".

Checks, gotchas & red flags

- **Never commit real or client data / credentials.** Use obviously fake names and amounts. A leaked key in git history is a hard interview fail — scrub with `git filter-repo` if it happens.
- **The number must be defensible.** If you claim "15 hours saved", be ready to explain the arithmetic. Vague "huge time savings" reads as fluff.

- **Show error handling**, not just the happy path — a reviewer looks for the break/exception branch. A recon that only handles perfect matches is a toy.
- **Don't over-engineer.** A four-tool project is impressive *only if each tool is justified*. If Power Query alone solves it, saying so shows better judgement than bolting on a bot.
- **Broken demo GIF / dead repo** undoes everything. Test the "how to run" steps on a clean machine.
- **README-less repo** = invisible work. If it isn't explained, it doesn't count.

Interview drill

Q1. "Walk me through automating the monthly GST reconciliation (GSTR-2B vs purchase register)." Clarify: inputs are the 2B JSON/Excel from the GST portal and the purchase register from Tally; monthly; a few thousand lines. Map: download 2B, match on GSTIN + invoice no + taxable value, classify matched / mismatch / missing-in-books / missing-in-2B. Tool: Power Query for the match (auditable, no-code, finance-owned) with a Python fallback for volume; RPA only for the portal download since there's no clean API. Exceptions: rounding tolerance of ₹1, fuzzy invoice numbers flagged for human review — maker-checker before any ITC is claimed. Govern: no ITC auto-claimed, full log. Measure: cuts a two-day manual match to an hour, and catches mismatches that risk ITC reversal.

Q2. "You have Excel, VBA, Power Query, Python and UiPath. A vendor emails an invoice PDF daily that must be keyed into SAP. Pick your stack." Extraction: the PDF has no API, so either UiPath Document Understanding or a Python parser (pdfplumber/camelot) to pull header + line items. Entry into SAP: SAP GUI has scripting, so UiPath (or the SAP GUI Scripting API) drives the transaction reliably. So: Python/UiPath for extract, UiPath unattended for SAP entry, with a validation table check before posting. VBA/Power Query don't fit — they can't read the PDF or drive SAP well. I'd add a maker-checker park-and-post so a human releases the document.

Q3. "What makes an automation *maintainable* six months later?" Clear naming and comments, externalised config (paths, emails, thresholds in one place, not hard-coded), robust selectors/keys instead of positions, error handling with logging, version control with a README, and no embedded credentials. Plus a documented owner — automation that only one person understands is a liability when they leave.

Learn/practise (free) + Cheat-Sheet

Tool decision cheat-sheet:

Job	Reach for	Why
Data cleanup, joins, refreshable ETL in Excel	**Power Query**	No-code, auditable, repeatable on refresh
Excel event/UI automation, custom functions	**VBA / Office Scripts**	Lives in the workbook, no extra licence
Heavy data, stats, APIs, ML, big files	**Python (pandas)**	Scales past Excel, huge library ecosystem
Reporting/dashboards from many sources	**Power BI**	Model + visuals + scheduled refresh
GUI-only legacy app, no API, repetitive	**RPA (UiPath)**	Mimics the human across apps
App has a clean API	**API integration**	Robust — prefer over RPA every time
Cross-app workflow glue, cloud connectors	**Power Automate**	Trigger-based, 100s of connectors

Excel / Power Query / VBA quick-reference:

- Excel functions: `XLOOKUP`, `SUMIFS`, `INDEX/MATCH`, `LET`, `LAMBDA`, `FILTER`, `UNIQUE`, `TEXTSPLIT`, `IFERROR`.
- Shortcuts: `Ctrl+T` (table), `Ctrl+Shift+L` (filter), `Alt+=` (AutoSum), `Ctrl+Shift+Enter` (legacy array), `F4` (lock `$` reference / repeat), `Ctrl+G→Alt+S` (Go To Special), `Alt+A+M` (remove duplicates).
- Power Query M essentials: `Table.Merge`, `Table.NestedJoin`, `Table.AddColumn`, `Table.Group`, `Table.SelectRows`, `Text.Trim`, `each`, `[Column]` referencing; parameters for file paths.
- VBA essentials: `Range`, `Cells`, `For Each`, `Workbooks.Open`, `Application.ScreenUpdating = False`, `On Error Resume Next` / `GoTo`, `Dim x As Long`.

Free practice: UiPath Community + Academy; Microsoft Learn (Power Query, Power BI, Power Automate); Kaggle for pandas datasets; the RBI/NSE public data for realistic finance files. Rehearsal loop: take any manual task you did this week, time it, automate it with the *simplest* tool that works, log the saving, push to GitHub. Ten such micro-projects, and the portfolio writes itself.